

Verifiable result traceability in AI research: a claim registry that links published numbers to their artifacts

Trazabilidad verificable de resultados en la investigación en IA: un registro de afirmaciones que vincula los números publicados con sus artefactos

Jonathan Herrera-Vásquez

Computer scientist. Department of Computer Science, Universidad Americana de Europa (UNADE), Cancún, Quintana Roo, Mexico.

jonnabio@gmail.com ORCID: [ORCID]

Abstract

Numbers in artificial intelligence (AI) research papers are usually copied by hand from analysis outputs into manuscripts, and when one study feeds a thesis, several papers and a book chapter, the same number is restated many times. Nothing then checks that each printed value still matches its source. This paper presents a claim registry, a single file that records each reported number with a resolver that re-derives it from committed artifacts and with every place where it is printed, and a verifier that fails the build on any mismatch, unregistered number or reuse of an unpublished result. We report a case study of its use in a doctoral research program on explainable AI. At a pinned commit the registry held 321 claims checked at 487 sites in four documents. We coded 58 defects recorded by audits, peer-style reviews and root-cause analyses between August and September 2026. Of the 30 defects that concerned a printed number, the current checks catch 16 and partly catch 13; of the 28 that concerned claim scope, coherence or reporting, they catch 3. The continuous-integration workflow failed every one of its first 37 runs, which shows that a check that never passes protects nothing. Mechanical verification removes numeric drift as a class of error but leaves overclaiming to human review.

Keywords: reproducibility; research software; artificial intelligence; continuous integration; scientific integrity; explainable AI.

Resumen

En la investigación en inteligencia artificial (IA), los números de los artículos suelen copiarse a mano desde los resultados del análisis hasta los manuscritos, y cuando un mismo estudio alimenta una tesis, varios artículos y un capítulo de libro, cada número se repite muchas veces. Nada verifica entonces que cada valor impreso siga coincidiendo con su fuente. Este artículo presenta un registro

de afirmaciones: un único archivo que asocia cada número publicado con un resolovedor que lo recalcula a partir de artefactos versionados y con cada lugar donde se imprime, junto con un verificador que detiene la compilación ante cualquier discrepancia, número no registrado o reutilización de un resultado no publicado. Se reporta un estudio de caso de su uso en un programa doctoral sobre IA explicable. En una versión fijada, el registro contenía 321 afirmaciones verificadas en 487 lugares de cuatro documentos. Se codificaron 58 defectos registrados por auditorías, revisiones y análisis de causa raíz entre agosto y septiembre de 2026. De los 30 defectos relativos a un número impreso, las verificaciones actuales detectan 16 y detectan parcialmente 13; de los 28 relativos al alcance de las afirmaciones, la coherencia o el reporte, detectan 3. El flujo de integración continua falló en sus primeras 37 ejecuciones, lo que muestra que una verificación que nunca pasa no protege nada. La verificación mecánica elimina la deriva numérica como clase de error, pero deja la sobreafirmación a la revisión humana.

Palabras clave: reproducibilidad; software de investigación; inteligencia artificial; integración continua; integridad científica; IA explicable.

1 Introduction

Concern about the reproducibility of computational research is long-standing [1–3], and it has reached artificial intelligence (AI) in particular [4–6]. Surveys of researchers report that most have failed to reproduce another group’s result [7]; analyses of machine-learning papers find that undocumented choices and data leakage can change conclusions [8, 9]. Responses have focused on what is released with a paper: code, data and checklists [10, 11], data that are findable and reusable [12], and notebooks that can be rerun [13, 14].

A smaller literature addresses a different failure: the number printed in the paper is not the number the analysis produced. In psychology, automated checks of reported test statistics found inconsistencies in about half of the papers examined [15], and arithmetic checks of reported means exposed further anomalies [16]. Literate programming and dynamic documents prevent this failure at its source by generating text and results together [17–19].

Dynamic documents fit a single report. They fit less well a research program in which one set of experiments feeds a thesis, several journal manuscripts and a book chapter, each written by hand in a different format and revised over months. In that setting a number is restated in many places; a correction made in one document does not reach the others; a value retired by an audit can reappear in a figure; and a result already published in one venue can be printed again in a submission to another. None of these is a failure of code release, so the reproducibility tools above do not detect it.

This paper describes a lightweight mechanism for that setting, a claim registry with an automated

verifier, and evaluates it in a case study. Its contributions are: (i) the design of the registry and its checks; (ii) an incident catalogue of 58 defects found in one doctoral program, coded by type and by whether the checks now detect them; and (iii) evidence on which kinds of error mechanical verification can and cannot remove.

2 Materials and methods

2.1 Research setting

The case is a doctoral research program on the evaluation of explainable AI methods for tabular classifiers. Its experiments write per-run result files and statistical exports to a git repository. Four documents report the results: a thesis (Quarto), a published journal article and a journal manuscript under review (both $\text{\LaTeX}$), and a book chapter (Markdown). The same quantities, such as effect sizes, means and run counts, appear in several of them.

2.2 The claim registry

The registry is one TOML file under version control. Its main entry is a *claim* (Fig. 1). A claim records an identifier, a *source* expression, the value as printed, a tolerance, and a list of *sites*. The source names a resolver, a small function that recomputes the value from committed artifacts. Each site gives a manuscript file and the literal text that prints the value there. Let a claim c have registered value v_c , tolerance τ_c , resolver r_c and sites S_c . Given the committed artifacts A , the claim passes when

$$|r_c(A) - v_c| \leq \tau_c \quad \text{and} \quad \forall s \in S_c : \text{text}(s) \subseteq \text{file}(s). \quad (1)$$

The first condition catches a result that changed after the text was written; the second catches a manuscript that changed without its result.

Four further entry types close gaps that per-claim checks leave open. A *retired* entry records a value that an audit removed; the verifier fails if that text reappears in any manuscript. An *unbacked* entry records a printed number that cannot be re-derived, with the reason, so that the gap is disclosed rather than hidden. *Coverage* lists manuscript files that must be fully registered: every numeric literal ℓ found in a covered file F must belong to

$$L(F) \subseteq L_{\text{reg}} \cup L_{\text{ret}} \cup L_{\text{unb}} \cup L_{\text{str}}, \quad (2)$$

the registered, retired, unbacked and declared structural literals (settings, thresholds, layout values). An unregistered number is therefore a failure, not a silence. *Exclusivity* lists files that must not print

any result registered for an unpublished manuscript, unless the same result is already published elsewhere.

2.3 Complementary checks

Five checks run beside the registry. Abstracts and keywords are generated from one source file into each manuscript, and a synchronisation check confirms that every manuscript includes them and that no generated file contains a control character. A shared-literal scan compares every decimal number in the manuscript under review with those in the published article, independently of the registry, to detect results reported twice. Source files reconstructed after a data loss are pinned by hash. The verifier is run from a fresh clone, because a file present on the author’s disk but not tracked by git passes locally and fails for everyone else. All checks run in continuous integration (CI) on every push.

2.4 Implementation

The verifier and the resolvers are plain Python with no dependency beyond the standard library and the libraries the analysis already uses. At the pinned commit the verifier had 276 non-blank lines, the resolvers 582, and the registry file 3,076. A verification run reads the registry, calls every resolver, scans every manuscript listed in a claim, in coverage or in exclusivity, and prints either one summary line or the list of failures with the file and line of each. A failure blocks the merge of the change that caused it, so a manuscript cannot be updated past a number that no longer matches. Adding a number to a manuscript therefore takes three steps: choose or write the resolver that re-derives it at the aggregation the text states, add the claim with its site, and print the number.

2.5 Case-study measurements

Registry composition and growth. The registry was read from git at a pinned commit, the state after the last refactor of the manuscript under review, and at each of the commits that changed it since its creation, to count claims, sites and the other entry types.

Incident catalogue. We collected every defect recorded in the program’s quality records between 11 August and 28 September 2026: one cross-document alignment audit, three peer-style rigor reviews, a coverage triage, a remediation plan, three root-cause analyses and one chapter synchronisation report. Each entry was coded on four variables: whether it concerns a printed number; its class (Fig. 3); how it was found (manual audit or review, building the registry, a coverage sweep, a fresh-clone run, or reading the rendered output); and whether the current checks would detect it if it were reintroduced (yes; partly, when the value is checked but its label, aggregation or context

is not; or no). Findings carried forward unchanged from an earlier review, and findings that are not defects in a document, were excluded. The author coded the catalogue with AI assistance and checked every entry (see Acknowledgments).

CI history. The run history of the CI workflow was exported from the repository host and summarised by outcome.

All measurements are scripts that read committed files; their outputs are themselves registered claims of this paper.

3 Results

3.1 Registry composition

At the pinned commit the registry held 321 claims checked at 487 sites in four documents (Table 1). 85 claims were printed in more than one document; these are the values most exposed to cross-document drift. Fifty distinct resolver kinds re-derive the values. The registry grew from 44 claims at 93 sites when it was created to its current size over 39 commits (Fig. 2), mostly in two bursts that followed audits.

Table 1: Composition of the claim registry at the pinned commit.

Entry	Count
Claims (re-derived values)	321
sites in the thesis	286
sites in the manuscript under review	146
sites in the book chapter	30
sites in the published article	18
Retired values	27
Unbacked values	10
Declared structural literals	69
Files under coverage	19
Files under exclusivity	13

3.2 Incident catalogue

The quality records yielded 65 entries, of which 58 were distinct defects in a document; 22 were rated major. Thirty concerned a printed number and 28 did not. Fig. 3 shows the defects by class and by whether the current checks detect them.

For numeric defects the checks are effective. Of the 30, 16 would be detected and 13 would be partly detected; one would not, a traceability table that cited an existing but wrong artifact,

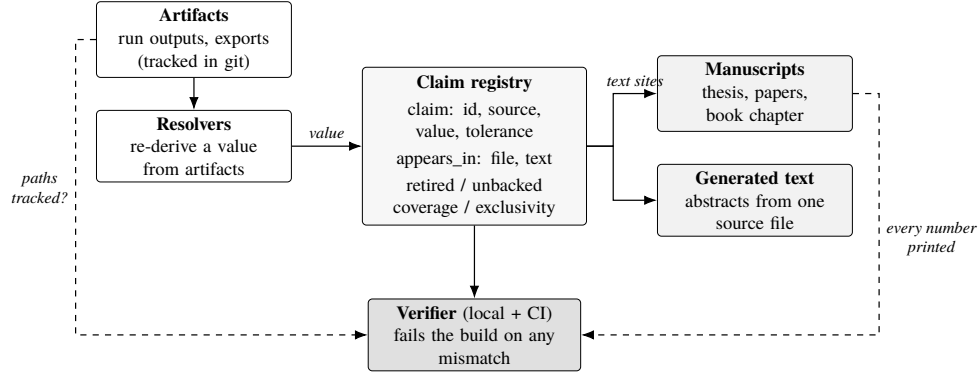


Figure 1: The verification loop. A resolver re-derives each registered value from tracked artifacts; the verifier compares it with the registered value and with the text at every manuscript site, and checks that cited artifacts are tracked.

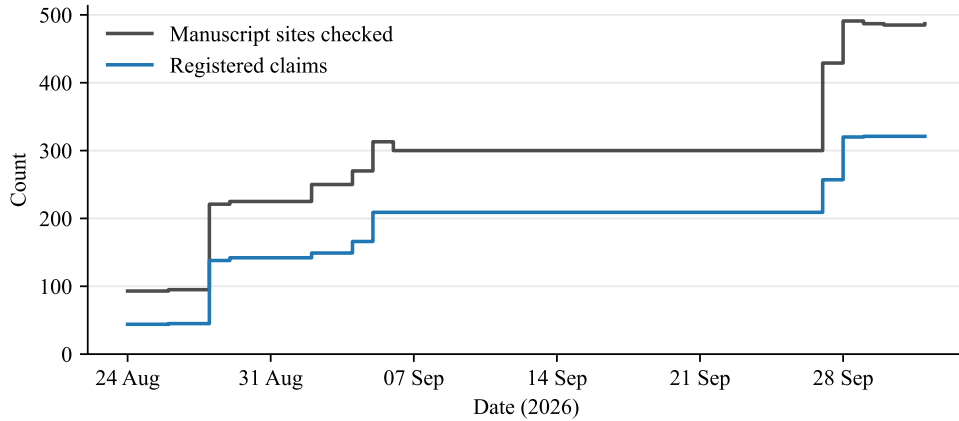


Figure 2: Growth of the registry: claims and checked manuscript sites at each day on which the registry changed.

which passes because the verifier checks that a cited file exists, not that it is the right file. The partly detected defects share a pattern: the value is right, and what is wrong is the text around it. Examples are a correct gap attributed to the wrong model, run-level means labelled as block-level means, and a sample size in a caption that belongs to a different statistic.

For non-numeric defects the picture reverses. Of the 28, the checks detect 3, all of them mechanical failures: an escape sequence that corrupted a generated abstract, a supplementary file missing from an upload bundle, and a verification input that existed on one machine only. The other 25 are claims stated more broadly than the evidence, internal contradictions between sections, and reporting slips. The largest single class, claim scope, is almost invisible to them: 12 of its 13 defects are not detected.

Most defects, 47 of the 58, were found by people reading: the audit and the reviews. Eight

defects were found by the tooling itself, while the registry was being built or extended: registering a value forced a comparison with its artifact, and the coverage sweep surfaced numbers that nobody had registered. The remaining three were found by reading the rendered output, which is where a broken cross-reference or a corrupted generated abstract becomes visible. Table 2 gives one example per class and the check, if any, that now responds to it.

Table 2: Example defects from the catalogue and the response of the current checks.

Class	Example	Current response
Value differs from artifact	A chapter printed draft figures that matched no artifact	Detected: the site text no longer matches the re-derived value
Cross-document inconsistency	Two manuscripts printed different values for one experimental cell	Detected: one claim carries the sites of both
Wrong label or aggregation	A correct gap attributed to the wrong model	Partly: the value is checked, the label is not
Value without an artifact	Two tables printed different values for one reference condition	Partly: the number must be declared unbacked, with a reason
Artifact provenance	Cited result files present on disk but not tracked by git	Detected: the fresh-clone run fails
Prior-publication overlap	Results of a published article printed again in a submission	Detected: the shared-literal scan fails
Rendering defect	An escape sequence corrupted a generated abstract	Detected: the synchronisation check fails
Claim scope	An ordering claimed for a model family that holds for one member only	Not detected

3.3 Continuous-integration history

The CI workflow ran 80 times. Its first 37 runs all failed. The causes recorded when the workflow was investigated were in its environment (an input file that was not tracked and a generated file that changed on every build), not in the manuscripts. For five months the check was red for reasons unrelated to what it was meant to protect, so its failures carried no information. After those causes were fixed, all 43 runs passed.

4 Discussion

The case supports a narrow conclusion. A registry that binds each printed number to its artifact and to every place it appears removes numeric drift as a class of error: a stale value, a retired value that comes back, a number that differs between two documents, an unregistered number and a result reported twice all fail the build. These were the defects that the August audit had found by reading.

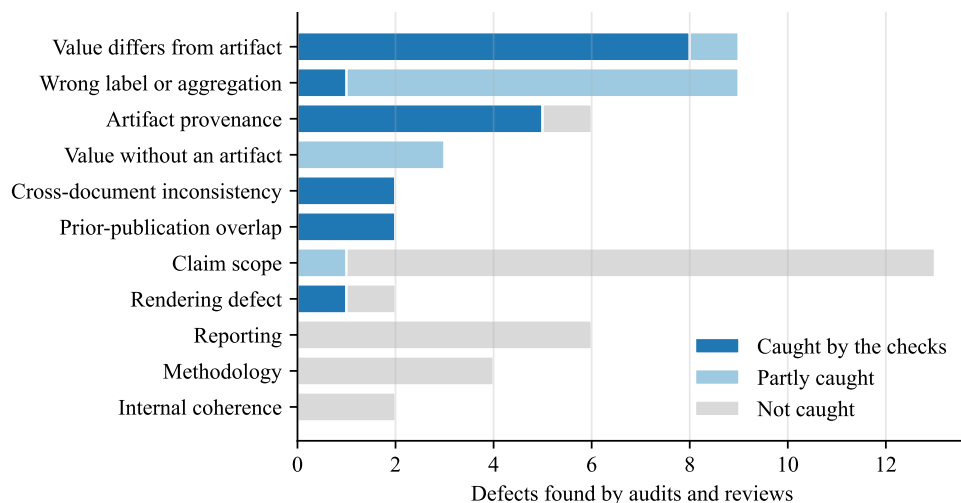


Figure 3: The 58 defects by class and by whether the current checks detect them.

Three lessons concern what the mechanism does not do. First, checking a value is not checking its meaning. The partly detected defects show that a correct number attached to the wrong label passes; binding the label or the aggregation to the site text, rather than the number alone, would close part of this gap. Second, about half of all defects were about claims, not numbers. Scope, coherence and reporting need human review, and the registry helps only indirectly, by freeing reviewers from re-deriving numbers. Third, a check must be kept green to be useful: a workflow that fails for unrelated reasons trains its users to ignore it.

Limitations. This is one program, coded by its author, who also built the tooling; the coding of “detected now” was not checked by a second rater and may favour the tool. The catalogue covers only the defects that audits and reviews recorded, so the rate of undetected defects is unknown. The registry adds work: every new number must be registered with a resolver before it is printed.

Relation to existing tools. Automated consistency checks of reported statistics [15, 16] test whether numbers in a published paper agree with each other; they need no access to the data, but they cannot tell whether a value is the one the analysis produced. Dynamic documents [18, 19] guarantee that correspondence within one document, at the price of writing the document inside the analysis environment. The registry sits between the two: manuscripts stay in whatever format each venue requires and are written by hand, and the correspondence with the artifacts is checked from outside. It also checks a property neither approach addresses, that a result is not reported twice across venues.

Recommendations. For research programs that report the same results in several documents: register each reported number once, with every site; record retired values so they cannot return; treat an unregistered number in a finished manuscript as an error; check prior publication by comparing

numbers, not only text; run the checks from a fresh clone and in CI; and keep human review for claims.

5 Conclusions

A claim registry is a small, practical addition to the reproducibility toolkit for AI research. In the case studied it re-derived 321 reported values at 487 sites and would now detect almost all of the numeric defects that manual audits had found. It does not detect overclaiming, which was as common as numeric error. Release of code and data shows that a result can be reproduced; a registry shows that the paper reports the result that was produced.

Data availability

The registry, the verifier, the measurement scripts and the incident catalogue are in the public repository <https://github.com/jonnabio/xai-eval-framework>, archived at <https://doi.org/10.5281/zenodo.23111684>.

Acknowledgments

The author conceived, designed and directed this work. Generative AI tools (Anthropic Claude) were used as assistants under the author’s direction for programming support, first-pass coding of the incident catalogue (checked and approved by the author), the search for related literature (every reference was verified against its DOI record and approved by the author), and language editing. The author takes full responsibility for the content.

References

- [1] R. D. Peng, “Reproducible research in computational science,” *Science*, vol. 334, no. 6060, p. 1226–1227, Dec. 2011.
- [2] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, “Ten simple rules for reproducible computational research,” *PLoS Computational Biology*, vol. 9, no. 10, p. e1003285, Oct. 2013.
- [3] V. Stodden, M. McNutt, D. H. Bailey, E. Deelman, Y. Gil, B. Hanson, M. A. Heroux, J. P. Ioannidis, and M. Taufer, “Enhancing reproducibility for computational methods,” *Science*, vol. 354, no. 6317, p. 1240–1241, Dec. 2016.

- [4] M. Hutson, “Artificial intelligence faces reproducibility crisis,” *Science*, vol. 359, no. 6377, p. 725–726, Feb. 2018.
- [5] O. E. Gundersen and S. Kjensmo, “State of the art: Reproducibility in artificial intelligence,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.
- [6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, Apr. 2018.
- [7] M. Baker, “1,500 scientists lift the lid on reproducibility,” *Nature*, vol. 533, no. 7604, p. 452–454, May 2016.
- [8] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, p. 100804, Sep. 2023.
- [9] Z. C. Lipton and J. Steinhardt, “Troubling trends in machine learning scholarship: Some ml papers suffer from flaws that could mislead the public and stymie future research.” *Queue*, vol. 17, no. 1, p. 45–77, Feb. 2019.
- [10] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://jmlr.org/papers/v22/20-303.html>
- [11] B. J. Heil, M. M. Hoffman, F. Markowetz, S.-I. Lee, C. S. Greene, and S. C. Hicks, “Reproducibility standards for machine learning in the life sciences,” *Nature Methods*, vol. 18, no. 10, p. 1132–1135, Aug. 2021.
- [12] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. ’t Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, and B. Mons, “The fair guiding principles for scientific data management and stewardship,” *Scientific Data*, vol. 3, no. 1, Mar. 2016.
- [13] A. Rule, A. Birmingham, C. Zuniga, I. Altintas, S.-C. Huang, R. Knight, N. Moshiri, M. H. Nguyen, S. B. Rosenthal, F. Pérez, and P. W. Rose, “Ten simple rules for writing and sharing

computational analyses in jupyter notebooks,” *PLOS Computational Biology*, vol. 15, no. 7, p. e1007007, Jul. 2019.

- [14] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, “A large-scale study about quality and reproducibility of jupyter notebooks,” in *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, May 2019, p. 507–517.
- [15] M. B. Nuijten, C. H. J. Hartgerink, M. A. L. M. van Assen, S. Epskamp, and J. M. Wicherts, “The prevalence of statistical reporting errors in psychology (1985–2013),” *Behavior Research Methods*, vol. 48, no. 4, p. 1205–1226, Oct. 2015.
- [16] N. J. L. Brown and J. A. J. Heathers, “The grim test: A simple technique detects numerous anomalies in the reporting of results in psychology,” *Social Psychological and Personality Science*, vol. 8, no. 4, p. 363–369, Oct. 2016.
- [17] D. E. Knuth, “Literate programming,” *The Computer Journal*, vol. 27, no. 2, p. 97–111, Feb. 1984.
- [18] F. Leisch, *Sweave: Dynamic Generation of Statistical Reports Using Literate Data Analysis*. Physica-Verlag HD, 2002, p. 575–580.
- [19] R. Gentleman and D. Temple Lang, “Statistical analyses and reproducible research,” *Journal of Computational and Graphical Statistics*, vol. 16, no. 1, p. 1–23, Mar. 2007.